

DevOps - Bachelor 3 Developpement 2025/2026

TP1, TP2, TP3 Docker et Docker Compose

Document adapte aux notions vues en piscine B2

Partie	Sujet	Notions principales
TP1	Reparer une image Docker cassee	Dockerfile, layers, securite simple
TP2	Docker Compose: stack multi-services	Compose, volumes, .env, Adminer
TP3	Conteneurisation de TaskFlow	Dockerfiles, Compose, reseaux, Redis

Travail : individuel

Materiel : Docker, Docker Compose, un editeur de code, un terminal.

Important : ce document reste centre sur Docker et Docker Compose.

Perimetre et progression

Les étudiants ont déjà manipulé les images, les containers, les ports, les volumes, les réseaux, les variables d'environnement et Docker Compose. Les seules notions nouvelles indispensables sont expliquées directement dans les consignes.

Notions nouvelles ajoutées car nécessaires

- `.dockerignore` : permet de ne pas envoyer `node_modules`, `.env`, `.git` ou les logs dans le contexte de build Docker.
- Cache Docker : copier `package*.json` avant le code permet de ne pas réinstaller les dépendances à chaque modification de fichier source.
- Utilisateur non-root : un container ne doit pas forcément tourner avec les droits root. Pour une image Node officielle, l'utilisateur `node` existe déjà.
- Stabilisation Compose : si une API démarre avant sa base de données, elle peut planter. On peut utiliser `restart: on-failure` ou un `healthcheck`.

TP1 - Reparer une image Docker cassée

Seance 1 - Docker avance

Durée estimée : 1h30

Theme : Dockerfile, layers, sécurité simple, bonnes pratiques de build

Travail : individuel

Contexte

Un développeur junior a écrit le Dockerfile fourni pour containeriser une petite application Node.js. L'image peut builder et l'application peut fonctionner, mais le Dockerfile n'est pas acceptable pour une utilisation propre en production.

Votre mission : identifier les problèmes du Dockerfile, expliquer leurs conséquences, puis corriger l'image sans modifier le code de l'application.

Fichiers fournis

```
tp1/
├── Dockerfile      # fichier à analyser et corriger
├── index.js        # code de l'app - ne pas modifier
├── package.json    # dépendances du projet - ne pas modifier
└── tp1-enonce.docx # sujet initial, selon archive fournie
```

Notion nécessaire - `.dockerignore`

Le fichier `.dockerignore` fonctionne comme un `.gitignore`, mais pour Docker. Il évite d'envoyer des fichiers inutiles ou sensibles dans le contexte de build.

Notion nécessaire - cache Docker

Docker réutilise les layers si les fichiers copiés n'ont pas changé. Pour une application Node, on copie d'abord `package*.json`, on installe les dépendances, puis on copie le code.

Notion nécessaire - utilisateur non-root

Les images Node officielles contiennent déjà un utilisateur node. L'objectif est que le container n'exécute pas l'application en root.

Niveau 1 - Analyse - 30 min - Facile

Sans modifier le projet, identifiez les problèmes dans le Dockerfile fourni. Pour chaque problème trouvé, répondez aux trois questions suivantes :

1. Quelle ligne est en cause ?
2. Quel est le problème exact ?
3. Quelle est la conséquence en production ?

Il y a au moins 5 problèmes distincts à trouver.

A compléter	Diagnostic	Risque	Correction
Ligne(s)	Problème observé	Conséquence	Correction envisagée
...	...	...	...
...	...	...	...
...	...	...	...

Niveau 2 - Réparation - 45 min - Intermédiaire

Reécrivez le Dockerfile en corrigeant les problèmes identifiés. Vous ne devez pas modifier index.js ni package.json.

Objectifs à atteindre

- L'image finale doit faire moins de 200 MB.
- Aucun secret ne doit être présent dans l'image.
- Le container ne doit pas tourner en root.
- Le cache Docker doit être exploité correctement.
- Seuls les outils strictement nécessaires doivent être présents.
- L'application doit rester accessible sur le port 3000.

Vérification

```
# Builder votre image corrigée
docker build -t tp1:corrige .

# Vérifier la taille
docker images tp1:corrige

# Vérifier que l'app tourne
docker run --rm -p 3000:3000 tp1:corrige

# Ouvrir http://localhost:3000

# Vérifier que le container ne tourne pas en root
```

```
docker run --rm tp1:corrige whoami  
# Attendu : node  
  
# Verifier que les secrets n'apparaissent pas dans l'environnement de l'image  
docker inspect tp1:corrige --format '{{range .Config.Env}}{{println .}}{{end}}' | grep -iE  
"API_KEY|DB_PASSWORD|DB_HOST"  
# Attendu : aucune sortie
```

Criteres de reussite TP1

Critere	Verification
Diagnostic	Au moins 5 problemes expliques avec ligne, probleme, consequence.
Build	docker build -t tp1:corrige . fonctionne.
Taille	Image finale inferieure a 200 MB.
Securite simple	Pas de secrets dans le Dockerfile ou dans l'environnement de l'image.
Utilisateur	docker run --rm tp1:corrige whoami affiche node.
Fonctionnel	localhost:3000 affiche l'application.
.dockerignore	complet

TP2 - Docker Compose : stack multi-services

Seance 1 - Docker Compose

Theme : Docker Compose, volumes, .env, Adminer

Travail : individuel

Contexte

Vous recuperez une application decoupee en 3 services deja containerises. Les Dockerfiles de l'API et du frontend sont fournis. Les services existent, mais ils ne sont pas encore lances ensemble.

Votre mission : ecrire le docker-compose.yml qui fait fonctionner les services ensemble, persister les donnees PostgreSQL, puis ajouter Adminer pour administrer la base dans le navigateur.

Structure du projet

```
tp2/
├── api/
│   ├── index.js      # API Node.js qui lit/ecrit dans PostgreSQL
│   ├── package.json
│   └── Dockerfile
├── frontend/
│   ├── index.html    # interface web qui parle a l'API
│   └── Dockerfile
└── docker-compose.yml # a creer
```

Ce que fait l'application

- Le frontend, servi par Nginx, affiche une page web avec un formulaire.
- La page envoie les messages a l'API via des appels HTTP.
- L'API Node.js stocke les messages dans PostgreSQL.
- Les messages doivent persister apres un redemarrage des containers.

Variables d'environnement attendues par l'API

Variable	Valeur attendue
DB_HOST	Nom du service PostgreSQL dans Compose, attendu : database
DB_PORT	5432
DB_NAME	tp2db
DB_USER	tp2user
DB_PASSWORD	A definir dans le fichier .env local
PORT	3000

Notion necessaire - navigateur et reseau Docker

Le frontend s'execute dans le navigateur de votre machine. Dans le fichier fourni, il appelle l'API sur <http://localhost:3000>.

Notion necessaire - depends_on ne veut pas dire pret

depends_on demarre les containers dans le bon ordre, mais n'attend pas forcement que PostgreSQL soit totalement pret.

Pour éviter une API qui plante au démarrage, ajoutez au minimum `restart: on-failure` sur le service api. Un healthcheck PostgreSQL est proposé en bonus guide.

Etape 1 - Docker Compose - 30 min - Facile

Créez un fichier `docker-compose.yml` à la racine du dossier `tp2/`.

Contraintes

- Les 3 services doivent être définis : api, database, frontend.
- database utilise une image PostgreSQL officielle.
- database n'est pas accessible depuis l'extérieur : pas de ports sur ce service.
- frontend est accessible sur `http://localhost:8080`.
- api est accessible sur `http://localhost:3000`.
- api communique avec database via le nom de service database.
- Ajouter `restart: on-failure` sur api pour stabiliser le démarrage.

Vérification

```
docker compose up --build

# Ouvrir http://localhost:8080
# La page s'affiche

# Ouvrir http://localhost:3000/health
# Attendu : {"status":"ok"}

# Envoyer un message depuis le frontend
# Il apparaît dans la liste
```

Etape 2 - Volumes + .env - 30 min - Intermédiaire

2a - Volume pour la base de données

Ajoutez un volume nommé pour que les données PostgreSQL survivent à un `docker compose down` classique.

Vérification

```
# Ajouter 3 messages depuis le frontend, puis :
docker compose down
docker compose up -d

# Ouvrir http://localhost:8080
# Les messages sont toujours là
```

2b - Fichier .env pour les secrets

Aucun mot de passe ne doit apparaître en clair dans le `docker-compose.yml`. Les valeurs sensibles passent par un fichier `.env` local.

Verification

```
grep -i password docker-compose.yml  
# Attendu : uniquement des references comme ${DB_PASSWORD}, jamais la valeur reelle
```

Etape 3 - Service tiers : Adminer - 30 min - Intermediaire

Adminer est une interface web legere pour visualiser et administrer une base de donnees dans le navigateur. Vous devez l'ajouter comme 4e service de la stack.

Contraintes

- Utiliser l'image officielle adminer disponible sur Docker Hub.
- Adminer est accessible sur `http://localhost:8081`.
- Adminer doit pouvoir se connecter a database via le reseau Docker interne.
- database reste non exposee directement sur l'hote.
- Vous pouvez ajouter `ADMINER_DEFAULT_SERVER=database` pour pre-remplir le serveur.

Verification

```
# Ouvrir http://localhost:8081  
# L'interface Adminer s'affiche  
  
# Connexion :  
# Systeme      : PostgreSQL  
# Serveur      : database  
# Utilisateur  : valeur de DB_USER  
# Mot de passe : valeur de DB_PASSWORD  
# Base         : valeur de DB_NAME  
  
# La table messages est visible avec les donnees
```

Criteres de reussite TP2

Critere	Verification
Frontend accessible	<code>http://localhost:8080</code> affiche la page.
API accessible	<code>http://localhost:3000/health</code> repond <code>{"status":"ok"}</code> .
Communication API - BDD	Envoyer un message depuis le frontend : il s'affiche.
BDD non exposee	<code>localhost:5432</code> ne repond pas car aucun port n'est publie pour database.
Adminer accessible	<code>http://localhost:8081</code> affiche l'interface Adminer.
Adminer connecte a la BDD	La table messages est visible dans Adminer.
Donnees persistantes	Les messages survivent a <code>docker compose down</code> puis <code>docker compose up -d</code> .
Pas de secrets en dur	<code>grep password docker-compose.yml</code> ne montre pas le vrai mot de passe.

TP3 - Conteneurisation de TaskFlow

Theme : Dockerfiles, Docker Compose, reseaux, volumes, .env

Travail : individuel

Contexte et mission

Vous rejoignez une equipe en tant que DevOps Engineer junior. On vous confie TaskFlow, une application de gestion de taches. Elle tourne actuellement sur la machine d'un developpeur, sans lancement standardise avec Docker Compose.

Votre mission dans ce TP3 : conteneuriser l'application pour qu'une personne qui clone le repo puisse lancer toute la stack avec une seule commande.

Objectif final

Depuis la racine du projet, la commande suivante doit builder et lancer toute l'application :

```
docker compose up --build
```

Aucune installation de Node, Redis ou Nginx ne doit etre necessaire sur la machine de la personne qui clone le repo. Docker et Docker Compose suffisent.

La stack

Service	Image de base	Role
frontend	nginx:alpine	Sert l'interface web statique sur le port 80.
backend	node:20-alpine	API REST, logique metier et acces aux donnees.
cache	redis:7-alpine	Stockage/persistence des taches ou des stats selon l'application.

Regle importante - exposition des services
Le frontend et le backend peuvent etre accessibles depuis votre machine pour tester dans le navigateur.
Le cache Redis ne doit pas etre accessible depuis l'exterieur. Il doit etre joint uniquement par le backend via le nom de service cache.

Structure attendue du repo a la fin

Fichier / dossier	Contenu
backend/Dockerfile	Image Docker du backend Node.js.
frontend/Dockerfile	Image Nginx du frontend.
docker-compose.yml	Les services frontend, backend, cache, volumes et reseaux.
.env.example	Template des variables d'environnement sans vraies valeurs sensibles.
.dockerignore	Fichiers exclus du contexte Docker.

Etape 1 - Dockerfile backend - 35 min

Creez un Dockerfile dans backend/. Il doit permettre de lancer l'API Node.js dans un container.

Points attendus

- Utiliser une image Node alpine, par exemple node:20-alpine.
- Definir un WORKDIR, par exemple /app.
- Copier package*.json avant le reste du code pour profiter du cache Docker.

- Installer les dependances avec npm install. Si le projet contient un package-lock.json, vous pouvez utiliser npm ci.
- Copier le code du backend.
- Exposer le port utilise par l'API, par exemple 3001 si le backend ecoute sur 3001.
- Definir une commande de demarrage claire, par exemple npm start ou node server.js selon le projet fourni.

A ne pas faire

- Ne pas mettre de mot de passe ou de secret dans le Dockerfile.
- Ne pas copier node_modules depuis votre machine.
- Ne pas installer d'outils systeme inutiles.

Etape 2 - Dockerfile frontend - 20 min

Creez un Dockerfile dans frontend/. Il doit servir l'interface web avec Nginx.

Points attendus

- Utiliser nginx:alpine.
- Copier les fichiers statiques du frontend dans le dossier servi par Nginx.
- Si une configuration Nginx est fournie dans le projet, la copier au bon endroit.
- Exposer le port 80 dans le Dockerfile.

Etape 3 - docker-compose.yml - 40 min

Creez un docker-compose.yml a la racine du projet. Il doit lancer les 3 services et les connecter correctement.

Services attendus

- frontend : build depuis ./frontend, port publie sur votre machine, par exemple 8080:80.
- backend : build depuis ./backend, port publie sur votre machine, par exemple 3001:3001.
- cache : image redis:7-alpine, aucun port publie sur votre machine.

Contraintes Compose

- Le backend doit recevoir les variables necessaires pour joindre Redis : REDIS_HOST=cache et REDIS_PORT=6379, ou les noms exacts attendus par l'application.
- Les valeurs configurables doivent venir du fichier .env local ou avoir une valeur par default raisonnable.
- Redis doit utiliser un volume nomme pour conserver ses donnees si l'application les persiste.
- Les services doivent partager au moins un reseau Docker pour communiquer entre eux.
- Ajouter depends_on entre backend et cache si le backend a besoin du cache au demarrage.

Verification

```
docker compose up --build

docker compose ps
# Les 3 services doivent etre Up

# Tester le frontend
# Ouvrir http://localhost:8080

# Tester le backend si une route /health existe
```

```
curl http://localhost:3001/health

# Verifier que Redis n'est pas expose directement
# Il ne doit pas y avoir de ports: sur le service cache dans docker-compose.yml
```

Etape 4 - .env.example et .dockerignore - 15 min

.env.example

Creez un .env.example a la racine du projet. Il sert de modele pour les variables necessaires, sans vraie valeur sensible.

```
# Exemple a adapter aux noms reels attendus par l'application
APP_ENV=local
BACKEND_PORT=3001
REDIS_HOST=cache
REDIS_PORT=6379
REDIS_PASSWORD=change-me-if-needed
```

.dockerignore

Creez un .dockerignore pour eviter de copier des fichiers inutiles ou sensibles dans les images.

```
node_modules/
.env
*.log
.git/
dist/
build/
```

Si dist/ ou build/ sont necessaires pour votre image, ne les excluez pas. Adaptez le fichier a la structure du projet.

Criteres de reussite TP3

Critere	Verification
Commande unique	docker compose up --build lance la stack depuis la racine.
Frontend	Interface accessible dans le navigateur.
Backend	API accessible sur le port prevu, route /health si elle existe.
Cache interne	Redis n'est pas expose sur localhost, seul le backend l'utilise.
Persistence	Un volume nomme est declare pour Redis si l'application y stocke les donnees.
Configuration	.env.example present, pas de secret en dur dans docker-compose.yml.
Contexte Docker	.dockerignore present et coherent.

Rendu TP3

- Repo ou archive contenant backend/Dockerfile et frontend/Dockerfile.
- docker-compose.yml a la racine.
- .env.example a la racine, sans vraies valeurs sensibles.
- .dockerignore a la racine ou dans les contextes de build utilises.

Annexe - Commandes utiles

Docker

```
docker build -t nom:tag .
docker images
docker run --rm -p 3000:3000 nom:tag
docker run --rm nom:tag whoami
docker logs <container>
docker exec -it <container> sh
docker history nom:tag
docker inspect nom:tag
```

Docker Compose

```
docker compose up --build
docker compose up -d
docker compose ps
docker compose logs -f
docker compose logs -f <service>
docker compose exec <service> sh
docker compose down
docker compose down -v
```

Aide memoire - ports

Exemple	Signification
8000:80	Port 8000 sur votre machine -> port 80 dans le container.
3000:3000	Port 3000 sur votre machine -> port 3000 dans le container.
Pas de ports:	Le service est joignable uniquement par les autres containers du reseau Docker.

Aide memoire - volumes

```
services:
  database:
    image: postgres
    volumes:
      - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Aide memoire - variables .env dans Compose

```
.env
DB_USER=tp2user
DB_PASSWORD=change-moi
```

docker-compose.yml

environment:

POSTGRES_USER: \${DB_USER}

POSTGRES_PASSWORD: \${DB_PASSWORD}